Exam Questions

Big Data Processing (DPR.1) — FH Kufstein Tirol, WS 2026/27

Compiled from the course notebooks. The questions are ordered by **course progression** (Session 1 → Session 4), so you can study front to back. Every answer uses the measured values stored in the notebooks where available.

> **Duplicate note:** the Session 2 Exit Ticket asks exactly the same four Spark questions as Session 4 · Lab D, Part 1. They are merged into **Part 2** below and marked with both sources.

Contents

1. **Session 1 — Python concurrency & single-machine engines** (Q1–Q10)
   - 1.1 Warm-up: threads vs. processes (Q1–Q2)
   - 1.2 Do you really need a cluster? (Q3–Q6)
   - 1.3 Bonus: Python without the GIL (Q7–Q10)
2. **Spark fundamentals** — Session 2 Exit Ticket + Session 4 · Lab D Part 1 (Q11–Q14)
3. **Session 4 · Lab B — CSV vs. pickle vs. Parquet** (Q15–Q20)
4. **Session 4 · Lab C — DuckDB** (Q21–Q26)
5. **Session 4 · Lab D Part 2 — Spark ML** (Q27–Q34)
6. **Topic index** (quick lookup by concept)

Part 1 — Session 1: Python concurrency & single-machine engines

1.1 Warm-up: threads vs. processes

Source: `session-01/00_warmup_threads_vs_processes.ipynb`, §1–§2

Q1. Threads and processes are equally fast for the chores. Why doesn't the GIL matter for waiting?

Because the chores only **wait** (`time.sleep`) — they do almost no Python bytecode work. The GIL only prevents two threads from executing Python bytecode at the same time; a thread **releases the GIL while it is blocked** in sleep/disk/network I/O, so the other threads can run during the wait. With serial execution the total is $8 + 2 + 4 =$ **14 s**; with threads or processes the total is the longest chore = **8 s**. Both achieve the same overlap. Processes are equally fast because waiting needs no interpreter parallelism at all — the GIL is simply irrelevant for I/O waits.

Q2. Why do threads bring no speed-up for the CPU-bound work? What limits the process speed-up?

`cpu_heavy` is **pure Python, CPU-bound** code. The GIL allows only **one thread at a time** to execute Python bytecode, so 4 threads simply take turns on one interpreter → speed-up **1.0×**, even on 12 cores. Processes each have their **own interpreter and GIL**, so they can run truly in parallel on separate cores. The process speed-up is limited by: - the number of jobs: `JOBS = 4`, so at most **4×**; - the number of **physical cores** — with `Thread(s) per core = 2`, two vCPUs share one physical core, and pure computation gains little from hyperthreading; - process startup and **IPC/pickling** overhead, memory bandwidth, and load imbalance.

So the realistic maximum is `min(JOBS, physical cores)`. (In the stored output the process run failed on spawn because the worker function was not importable from the notebook `__main__`; that is a separate macOS/Windows process-startup issue, not the GIL.)

1.2 Do you really need a cluster?

Source: `session-01/01_do_you_need_a_cluster.ipynb`, §8 — "typical exam questions"

Q3. Why are Polars and DuckDB faster even on the same single machine?

Because their heavy work runs in **native code (Rust/C++) outside the GIL** and uses **all CPU cores**; pandas is mostly single-threaded. They also use a **columnar (Arrow) layout** with vectorized/SIMD execution and cache-friendly access, **lazy query plans** with projection/predicate pushdown, and avoid per-row Python-object overhead. pandas is eager, loads all columns, and carries DataFrame/object overhead.

Q4. How many months of taxi data could this machine handle with DuckDB? What limits it?

DuckDB **streams from disk and spills to disk**, so RAM is not the hard limit. Roughly, each month is ~50 MB Parquet (~3M trips). A machine with `F` GB free disk can hold about `F / 0.05` months if kept as Parquet (e.g., 100 GB 2,000 months). A materialized `.duckdb` table is larger (in Lab C, 348 MB vs. 100 MB Parquet, 3.5×), so divide accordingly. **Disk capacity** sets the maximum volume; **CPU** sets the speed; **RAM** is a soft limit that affects how much can be processed without spilling.

Q5. Two situations where a single machine is not enough, even with DuckDB

1. **Data volume/compute exceeds one machine:** the data cannot fit on local disk or cannot be processed in acceptable time on one node's cores → distributed storage/compute (Spark).
2. **Concurrency, fault tolerance, or availability:** many concurrent writers/users, or the workload cannot tolerate a single point of failure. DuckDB allows only one writer, and one machine has no built-in redundancy.

(Also valid: streaming/real-time ingest, geographically distributed data, or cost-effective horizontal scaling.)

Q6. Big Data is not only Volume. Which other Vs did today's lab ignore completely?

Primarily **Veracity** (data quality: dirty values, duplicates, missing data — the lab does not analyze data quality) and **Velocity** (streaming/real-time — everything here is batch). It also largely ignores **Variety** (multiple source systems/semi-structured data; only one taxi table is used) and **Value** (business usefulness). So the lab focused on Volume and performance, not the other Vs.

1.3 Bonus: Python without the GIL

Source: `session-01/02_bonus_python_without_gil.ipynb`, §5

Q7. In the first two runs, 4 threads are no faster than 1 thread (speed-up 1.0×). Why?

Because the **GIL is still enabled in both runs**: - run 1 is the normal Python 3.12 build (the GIL can never be switched off); - run 2 is the free-threaded 3.14t build, but started with

`PYTHON_GIL=1`, which turns the GIL back on.

The benchmark is CPU-bound pure Python, so only one thread can hold the GIL and execute bytecode at a time; the four threads serialize. Note that the speed-up column compares **threads vs. serial within the same run**, not one run against another.

Q8. What limits the speed-up of run 3 (GIL off)?

With the GIL off, threads really run in parallel, but: - the benchmark uses `JOBS = 4`, so the absolute ceiling is **4×**; - the machine's **physical cores** are the real limit. If there are fewer than 4 physical cores, or if two "CPUs" are hyperthreads sharing one core (`Thread(s) per core = 2`), pure computation gains little from SMT and the speed-up stays below 4×; - thread startup, memory bandwidth, and uneven work distribution also reduce it.

So the practical maximum is `min(JOBS, physical cores)`.

Q9. Compare the serial times of run 1 and run 3. Which difference explains it — the Python version or the GIL? Why could removing the GIL cost something?

The **Python version** explains it. Python 3.11+ introduced a much faster interpreter (adaptive specializing interpreter, cheaper calls, etc.), so 3.14 executes the single-threaded benchmark faster than 3.12. Turning the GIL off does **not** speed up single-threaded code. Removing the GIL can cost something because the free-threaded build adds overhead — biased/per-object reference counting, deferred reference counting, object "immortality", and extra synchronization — so single-thread performance may be slightly slower (3.13 was experimental and had the larger penalty; 3.14 reduced it). In addition, C extensions not marked free-threading-safe **re-enable the GIL** and print a warning, and memory overhead is higher.

Q10. Polars, DuckDB and Spark were fast in the main lab with the normal Python build. How do they avoid the GIL problem without a free-threaded Python?

They do the heavy work in **native code outside the Python interpreter**, so Python threads are not the parallelism mechanism: - **Polars** is implemented in **Rust** (Arrow); **DuckDB** in **C++**. Both release the GIL while running their multi-threaded native kernels and use all cores; Python only builds the query and receives the result. - **Spark** executes on the **JVM**: PySpark sends the plan to JVM executors, and the data processing happens entirely in Java/Scala, outside Python. (Python UDFs are the exception — they ship rows to a Python process and are much slower.)

So there is no Python bytecode competing for the GIL; parallelism comes from native thread pools/JVMs/processes.

Part 2 — Spark fundamentals

Sources: `session-02/02_spark_fundamentals.ipynb` (Exit Ticket) **and** `session-04/04_spark_fundamental` (Part 1). Both notebooks ask the same four questions.

Q11. "Three lines of code, so Spark ran three operations immediately." Is she right? When does computation start?

No. `filter`, `select`, and `groupBy().count()` are all **transformations**. They only build the logical plan; `groupBy` is wide but still lazy. **Nothing executes until an action** such as `show`,

`count`, `collect`, or `write`. Catalyst optimizes the entire chain into one job when the action is called.

Q12. `groupBy("PULocationID").count()` on 8 partitions: why is a shuffle needed, and what do the two `HashAggregate` steps do?

`groupBy` is a **wide transformation**: the same `PULocationID` values are scattered across the 8 input partitions, so all rows with the same key must be brought to the same partition → an `Exchange` (**shuffle**) by `hash(PULocationID)`. - **First `HashAggregate` (partial):** runs locally in each input partition and pre-aggregates counts per key, reducing the amount of data that must be shuffled. - **Second `HashAggregate` (final):** runs after the shuffle and merges the partial counts into one final count per `PULocationID`.

With 8 partitions you get up to 8 partial aggregates, then the shuffle, then the final merge.

Q13. `df.filter(fare_amount > 50).show(10)` returns 10 rows. Did Spark read only 10 rows?

No. `show(10)` is an **action**; it starts a job and scans partitions/row groups, applying the pushed-down filter, until it has collected 10 rows (`take`), then stops. It does **not** read exactly 10 rows: it may read entire partitions or row groups, and how much it reads depends on data layout. Predicate pushdown and row-group min/max statistics let it skip data, but the number of rows read is not 10. This matters because `show`/`count` trigger real work; `limit` plus filters enable early stopping, while `count` reads everything.

Q14. Joining 3M trips with the 265-row zone table: why `BroadcastHashJoin`, what is the benefit, and what can go wrong?

The zone table is tiny (265 rows), far below `spark.sql.autoBroadcastJoinThreshold` (default **10 MB**). Spark therefore **broadcasts** it to every executor and builds an in-memory hash table there. - **Benefit vs. `SortMergeJoin`:** the 3M-row side does **not** need to be shuffled or sorted; only the small table is replicated. Much lower network and latency cost for a small×large join. - **Risk:** broadcasting a genuinely large table duplicates it on every executor → driver/executor **out-of-memory**, network saturation, or broadcast timeout. Only broadcast tables that are actually small (and raise the threshold only deliberately).

Part 3 — Session 4 · Lab B: CSV vs. pickle vs. Parquet

Source: `session-04/02_file_formats_and_pickle.ipynb`, §9

Q15. Three concrete disadvantages of CSV, with measured effects

1. **Much larger files.** CSV was **101 MB**; Parquet snappy **26 MB**, Parquet zstd **22 MB** (4–5× larger).
2. **Much slower I/O.** CSV write **2.71 s**, read **0.74 s**; Parquet write **0.19 s**, read **0.02 s** (14× slower write, 37× slower read).
3. **Types are lost.** After the CSV round trip, both datetime columns came back as `str`, and `int32` columns became `int64`; `category` and nullable `Int8` are not preserved. It also adds an unwanted `Unnamed: 0` index column unless `index=False`.
4. (Bonus) **No column pruning or predicate pushdown.** CSV must read all 19 columns and all rows; Parquet can read only the needed columns and skip row groups.

Q16. Which formats keep `category` and nullable integers?

pickle and Parquet keep them; CSV does not. Parquet stores the logical/physical schema; pickle stores the Python object and its dtype metadata. CSV is plain text with no schema, so pandas must re-infer types on every read and cannot recover `category`, nullable `Int8`, or datetimes — they degrade to `str`, `float64`/`int64`.

Q17. Why does Parquet scale better when only 3 of 19 columns of a 50 GB dataset are needed?

Because Parquet is **columnar**: it stores each column separately, so only the 3 required column chunks are read and decompressed. It also stores **row-group statistics (min/max)**, so whole row groups that cannot match a filter are skipped. The 50 GB file is never fully loaded. pickle is a single opaque blob: it must read, decompress, and deserialize **all rows and all columns** into RAM before any filtering — all-or-nothing. That is why pickle can be fast on small data but does not scale; speed on small data is not the same as scalability.

Q18. How to save the scikit-learn model for a server batch job in six months?

Save it with **joblib** (pickle-based, efficient for NumPy arrays), preferably as a bundle. Store at minimum: 1. the **scikit-learn version**, and 2. the **Python version**.

(The notebook's bundle also stores the feature list/order and a training-data description.) Reason: a pickle stores *references* to classes, not their code; a different scikit-learn/Python version can produce warnings, wrong behaviour, or errors.

Q19. Risk of `pickle.load` on `best_model.pkl` downloaded from a forum

Unpickling can **execute arbitrary code**: an object's `__reduce__` can name any callable, so loading a malicious pickle can run e.g. `os.system` and compromise the machine. The demo payload was only **109 bytes** and still ran code during loading. Rule: only load pickles you created or fully trust; share data as Parquet and models via safe formats such as ONNX or Spark's native save format.

Q20. True or false: "pickle is the best format for large datasets because it is fastest to read."

False. pickle is fastest only on small data because it is a near-raw memory dump and the whole file fits in RAM. For large datasets it must load everything (no projection, no predicate pushdown), is uncompressed by default, is Python-only, is version-fragile, and is unsafe. Parquet's columnar layout, compression, and row-group statistics are what scale.

Part 4 — Session 4 · Lab C: DuckDB

Source: `session-04/03_duckdb_database.ipynb`, §12

Q21. `duckdb.connect()` vs. `duckdb.connect("taxi.duckdb")` — what is the difference, and when do you use each?

- `duckdb.connect()` creates an **in-memory, transient** database. It disappears when the process ends. Use it for quick ad-hoc queries on Parquet files in one session.

- `duckdb.connect("taxi.duckdb")` opens/creates a **persistent single-file database**. Tables survive closing Python and can be reopened later. Use it for a local analytics database, a repeatable batch job, or sharing one database file.

Q22. Why is DuckDB called an "embedded" database? Name one advantage and one limitation.

It runs as a **library inside your Python process** — there is no separate database server. - **Advantage:** zero installation/administration, no network round trips, very fast in-process access, one portable file. - **Limitation:** only **one process may write** the file at a time (many readers are possible only in read-only mode). No concurrent multi-user writes, and it is confined to one machine.

Q23. Is one DuckDB file good for 30 analysts writing every few seconds?

No. DuckDB allows many readers **or one writer**; concurrent writes serialize/lock the file (the notebook demonstrates a second process failing with an `IOException` lock error). For many concurrent writers, use a **database server such as PostgreSQL** (or another client/server DB). A lakehouse/Spark is only needed if the data volume itself demands distribution.

Q24. `CREATE TABLE ... AS SELECT` vs. `VIEW` — what happens to each when new rows are inserted into `trips`?

- **CTAS table:** materializes and stores the result **inside the database** at creation time. New rows inserted into `trips` afterwards are **not** reflected; the table is a snapshot until rebuilt.
- **View:** stores only the **query text**, no data. It is always up to date and reflects new rows in `trips` immediately; it costs no disk space but is recomputed on each query.

Q25. `DELETE ...` inside a transaction, then `ROLLBACK` — how many rows does `trips` have, and which ACID property is demonstrated?

After `ROLLBACK`, `trips` has **5,972,150 rows** — the same as before the transaction. (Inside the transaction it had dropped to 5,119,705.) This demonstrates **Atomicity**: the transaction is all-or-nothing; the delete is completely undone.

Q26. Why is the `.duckdb` file the same order of magnitude as Parquet, while CSV would be much larger?

Measured: `.duckdb` **348 MB** vs. Parquet input **100 MB** (3.5× — same order of magnitude). Both store **columnar, compressed** data, so neither pays CSV's row-oriented, uncompressed text overhead. DuckDB's extra size comes from storing the 12 selected columns (including the computed `duration_min`) plus its own row/version metadata and compression choices. A CSV export would be several times larger because every value is serialized as uncompressed text, row by row (in Lab B, a 1M-row CSV was 101 MB vs. 22–26 MB Parquet, 4–5×).

Part 5 — Session 4 · Lab D Part 2: Spark ML

Source: `session-04/04_spark_fundamentals_and_ml.ipynb`, Exam-style questions, Part 2

Q27. `read.csv` returned instantly; `count()` took much longer. Why? What changes with `inferSchema=True`?

`read.csv` is a **transformation** — it returns a DataFrame lazily and reads no data; only the plan is created. `count()` is an **action**, so the file is actually scanned. With `inferSchema=True`, Spark must read the file once at `read.csv` time to infer column types, so the previously instant call itself becomes a job and takes longer. With an explicit schema, that inference pass is avoided and `read.csv` stays cheap.

Q28. Why is `fare_amount/total_amount` not allowed as a feature? Name the general problem.

Because they are only known **after the ride** (at drop-off; `total_amount` also includes tip, tolls, and extras). Using them to predict duration leaks the outcome: the model learns from information unavailable at prediction time and reports unrealistically good performance that will not hold in production. The general problem is **target/data leakage** (using post-outcome features).

Q29. Transformer or Estimator: `StringIndexer`, `OneHotEncoder`, `VectorAssembler`, `LinearRegression`? What does `Pipeline.fit()` return?

- `StringIndexer` → **Estimator** (fit learns the category→index mapping).
- `OneHotEncoder` → **Estimator** (fit produces an `OneHotEncoderModel`; in Spark 3+/4).
- `VectorAssembler` → **Transformer** (no fit; it just assembles columns).
- `LinearRegression` → **Estimator** (fit produces a `LinearRegressionModel`).

`Pipeline.fit()` returns a `PipelineModel` — a fitted Transformer containing all fitted stages; call `.transform(df)` on it.

Q30. Why does the linear model need one-hot encoding for `hour`, but gradient-boosted trees do not?

A linear model is linear in its inputs; one-hot encoding gives each category its own independent coefficient and avoids imposing a false numeric order (hour 23 is not "twice" hour 11.5, and weekday/IDs are labels). Trees split on **thresholds**, so they can model arbitrary non-linear/step effects and interactions directly on raw `hour`, `weekday`, or IDs; one-hot would only add sparse columns without giving the tree new information.

Q31. `CrossValidator` with 3×2 parameter values and `numFolds=5`: how many models are trained? Why tune on a sample?

$3 \times 2 \times 5 = 30$ model fits, plus one final refit of the best combination on the full training set **$= 31$ fits total**. (The notebook's own run used $2{\times}2{\times}3 = 12 + 1$.) Tuning on a sample because 30 fits on the full 12.7M-row dataset would take too long and consume too much memory; a sample still ranks the hyperparameters reliably at a fraction of the cost.

Q32. Teammate wants to `pickle.dump()` a Spark `PipelineModel`. What do you tell them?

Don't. A Spark `PipelineModel` is only a thin Python wrapper around objects that live in the **JVM**; Python pickle cannot serialize that JVM state and will not produce a portable, reloadable model. Use Spark's own format:

```
model.write().overwrite().save(path)
PipelineModel.load(path)
```

It writes a folder of JSON metadata + Parquet files and can be loaded from Python, Scala, or Java.

Q33. Spark 4 ANSI mode: what happens with `F.col("a") / F.col("b")` when `b == 0`, and how do you avoid it?

With ANSI mode enabled (default in Spark 4), division by zero throws a **divide-by-zero error** (`ArithmeticException` / `DIVIDE_BY_ZERO`) and fails the job, instead of returning `NULL`. Avoid it with: - `F.try_divide("a", "b")` → returns `NULL` on division by zero, - `F.when(F.col("b") != 0, F.col("a") / F.col("b"))`, - `F.col("a") / F.nullif("b", 0)`.

Disabling ANSI (`spark.sql.ansi.enabled=false`) is possible but not recommended.

Q34. 2 GB of data on a 16 GB laptop: Spark, DuckDB, or pandas + scikit-learn? Justify.

Not Spark. 2 GB fits easily in 16 GB, and Spark's JVM startup, shuffle, serialization, and scheduling overhead buy nothing on a single machine. - **DuckDB** is the best default for SQL/analytics: columnar, compressed, out-of-core, minimal setup, very fast on one node. - **pandas + scikit-learn** is fine if the workflow is Python ML and the data fits in RAM (2 GB does, though pandas may use several× that); it is simpler for scikit-learn pipelines. - **Spark** only when the data exceeds one machine or you must scale out/distribute.

Topic index

Topic	Questions
Python GIL, threads vs. processes	Q1, Q2, Q7–Q10
Single-machine engines (pandas, Polars, DuckDB)	Q3–Q6
Big Data 5 Vs	Q6
Spark lazy evaluation / transformations vs. actions	Q11, Q13, Q27
Spark shuffles and `HashAggregate`	Q12
Spark joins (`BroadcastHashJoin`)	Q14
File formats: CSV vs. pickle vs. Parquet	Q15–Q20
DuckDB (in-memory vs. file, views, ACID, concurrency)	Q21–Q26
Spark ML: leakage, Transformers/Estimators, tuning	Q27–Q34
Saving models safely (pickle vs. Spark format)	Q18, Q19, Q32
Choosing the right engine (Spark vs. DuckDB vs. pandas)	Q4, Q5, Q34

Source map

Part	Notebook	Section
1.1	`session-01/00_warmup_threads_vs_processes.ipynb`	§1–§2
1.2	`session-01/01_do_you_need_a_cluster.ipynb`	§8
1.3	`session-01/02_bonus_python_without_gil.ipynb`	§5
2	`session-02/02_spark_fundamentals.ipynb` + `session-04/04_spark_fundamentals_and_ml.ipynb`	Exercise 1 – Part 1
3	`session-04/02_file_formats_and_pickle.ipynb`	§9
4	`session-04/03_duckdb_database.ipynb`	§12
5	`session-04/04_spark_fundamentals_and_ml.ipynb`	Exercise 1 – Part 2